


Lec-9

* SQL in One video

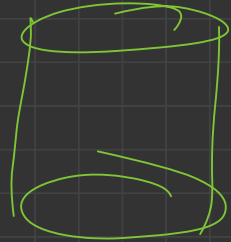
SQL $\Rightarrow$ Structured Query language

RDBMS $\Rightarrow$ MySQL, Oracle, MS access etc.

CRUD

SQL queries.

API
RDBMS

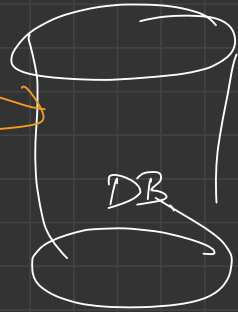


CRVD



Create
Read
Update
Delete.

20



RDBMS

RDBMS

RDBMS $\Rightarrow$ ① MySQL

② MSSQL server

⑤ Oracle

④ IBM.

→ Open some RDBMS.

→ Meta, Adobe etc.

SQL

- ① Query language.
- ② way to access data

MySQL

- ① MySQL itself a RDBMS
- ② CRUD done on it using SQL.

⇒ Install → ① MySQL community server
② MySQL workbench

2) workbench → DB create
Table

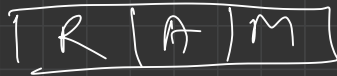
* SQL Data types

variable datatype

VARCHAR (255)

CHAR (255)

Arnd-
daten



$2\sqrt{5}$

SIGNED, unsigned

TINYINT $\rightarrow$ (-128 to 127)

UNSIGNED TINYINT $\Rightarrow$ (0 $\rightarrow$ 255)

create TABLE table-name (

col1 INT,

col2 INT UNSIGNED

);

⇒ Advanced DT :-

① JSON → JS Object Notation

Example - {
 "col1": "JSON",
}

SQL Types of Commands

① DDL

⇒ DQL ⇒

- ① DB
- ② table

user defined tables

Student

Dual table

↳ without using FROM

SELECT

wild cards

⇒

' % pa % '

→ 'abcpadb'

'pabc'

'adpa'

'_pa_'

apab

bpaC

Sorting

select * from worker ORDER BY salary;

ASC (Default)

DESC (↓)

* Distinct values

— Department (distinct)

HR
HR
Admin
Admin,
Account



HR
Admin
Accounts

* Data Grouping

→ find no. of employees working in different depart

→ HR → ?

Account → ?

Admin → ?

→ Grouping → Aggregation
(count)

⇒ GROUP BY → Aggregation fⁿ.
COUNT, SUM
AVG, MIN, MAX.

Worker

Name	Department
Monika	HR
Nam.	Admin
C	
D	HR
E	Admin
F	Admin
G	Account
H	Account
	Admin

Name	Department
Monika	HR
C	HR
"	Admin
"	Admin
"	Admin
"	Account
"	Account

Q find avg. salary per department?

* HAVING

→ select where to filter

"GROUP BY" HAVING

2) department, count, having More than 2 workers

* WHERE VS HAVING

* DDL Constants

① Primary Key Constraint

- Not Null
- Unique
- Only one P.K

Good practice

→ P.K INT

② Foreign Key

→ Fk refers P.K of other table.

work bench

③ Unique

```
create table Customer (  
    ;  
    Name VARCHAR(255) UNIQUE,  
    ;  
);
```

④ CHECK

```
create table account (  
    ;  
    CONSTRAINT acc_balance_chk CHECK (  
        balance > 0.00)  
    ;  
);
```

④ DEFAULT

create table account (

!

balance INT NOT NULL Default 0,

!.

)

* DML Data Modification Language.

* Referential Constraint

① INSERT

② Delete Constraint

→ ON Delete Cascade
→ ON Delete Set Null.

Replace :-

- ① Data already present, Replace.
- ② Data not present, INSERT

Replace Vs Update

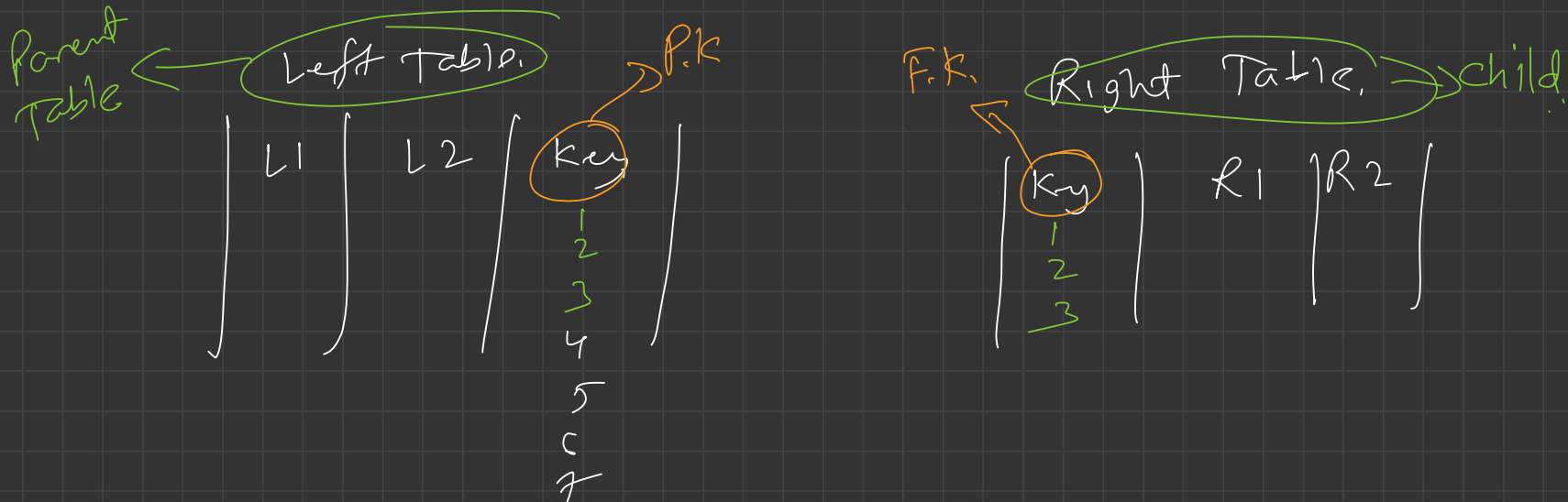
→ if row is not present, Replace will add a new row while UPDATE will do Nothing

#

JOINS

⇒ F.K → Relations

→ we → Data fetch → JOINS



① INNER JOIN

Resultant table

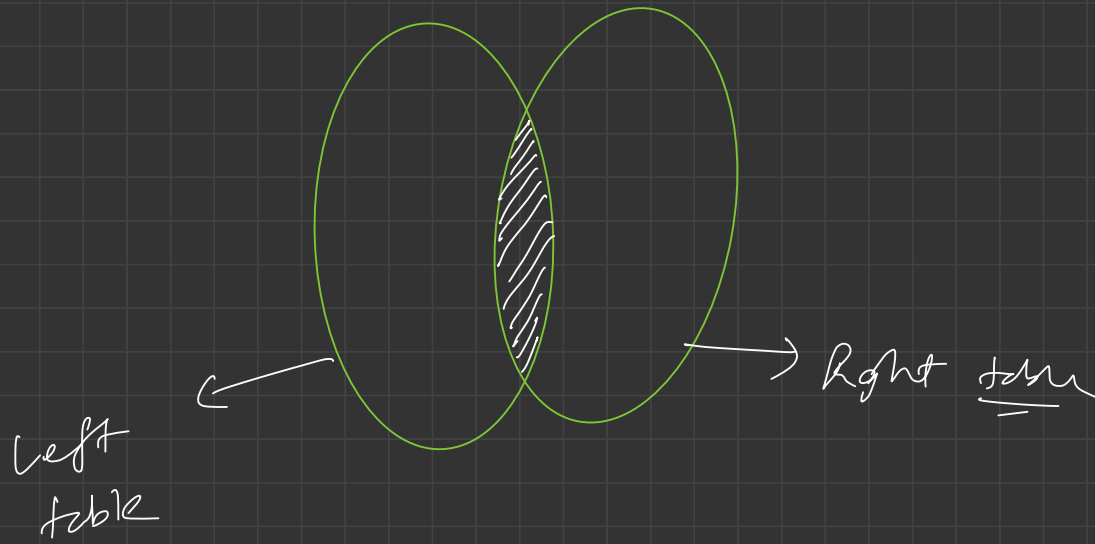
key	L1	L2	R1	R2
1				
2				
3				

— Join → To apply joins, there should be a common attribute.

Syntax!

Select C.*, O.* FROM Customer AS C INNER JOIN Orders AS O
ON c.id = o.cust-id;

Annotations:
- C.* points to "C.id, C.name."
- O.* points to "O.product."
- AS points to "aliasing".



INNER JOIN

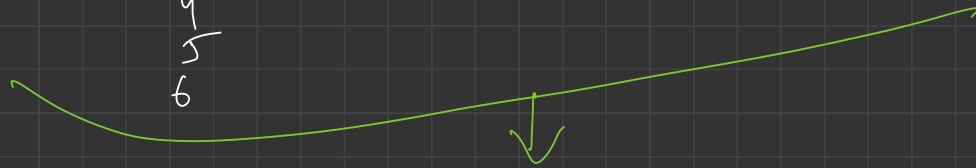
⑦ Left Join

Left Table

L1	L2	key
		1
		2
		3
		4
		5
		6

R Table

key	R1	R2
1		
2		
3		



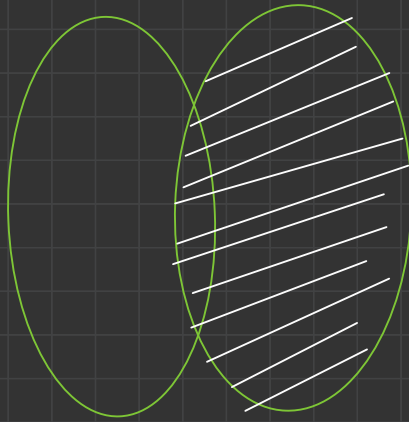
Resultant

key	L1	L2	R1	R2
1				
2				
3				
4			Null	Null
5			Null	Null
6				

select c.*, o.* FROM Customer as C LEFT JOIN
Orders as O
ON c.id = o.cust-id;

③ Right join

— intended in knowing only the
right side info

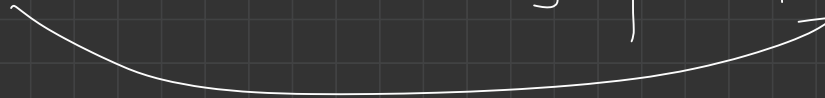


Left table

L1	L2	Key
		1
		2
		3

Right table

Key	R1	R2
1		
2		
3		
4		
5		



Result

Key	L1	L2	R1	R2
1				
2				
3				
4	Null	Null		
5	Null	Null		

④ Full join

Left

L1	L2	key
		1
		2
		3
		4
		5
		6

Right

key	R1	R2
1		
2		
3		
7		
8		
9		

MySQL Full JOIN X keyword X.

Simulated

Left Join $\cup$ Right Join

Syntax

select * from lefttable as l LEFT JOIN righttable as r
ON l.key = r.key.

UNION

select * from lefttable as l RIGHT JOIN righttable as r
ON l.key = r.key;

⊗ Cross join

→ Cartesian product

→

T_L
5 rows

T_R
10 rows

Result 1 - $5 \times 10 = 50$ rows

eg.

L_1	L_2
1	A
2	B

R_1	R_2
3	C
4	D

2) Result
=

→ Industrial use X.

L_1	L_2	R_1	R_2
1	A	3	C
1	A	4	D
2	B	3	C
2	B	4	D

⑤ Self Join

— emulate.

— 'INNER JOIN'

— Alias 'AS'

eg. select e1.id, e2.id, e2.name
FROM employee as e1
INNER JOIN
employee as e2
ON e1.id = e2.id;



→ Work bench senior
* Project

* EMPLOYEES

	id	empID	name	startdate	clientID
▶	1	1	A	2021-04-21	3
	2	2	B	2021-03-12	1
	3	3	C	2021-01-16	5
	4	3	D	2021-04-27	2
	5	5	E	2021-05-01	4
	NULL	NULL	NULL	NULL	NULL

[illegible]

* CLIENT

[illegible]

Q can we join w/o using JOIN keyword?

→ Yes

Syntax

```
select * FROM lefttable, Righttable WHERE  
lefttable.id  
= Righttable.id;
```

* Set Operations

{ 1, 2, 3, 4, 5, 6, 7, 8 }

Table 1

col 1	col 2
A	1
B	1
C	2

Table 2

col 1	col 2
A	1
B	2
D	3

① Union

② Intersection

③ MINUS

→ UNION
=

$T_1 \cup T_2$

$T_1 \cup T_2 \rightarrow$

col 1	col 2
A	1
B	1
C	2
B	2
D	3

— ROWS.

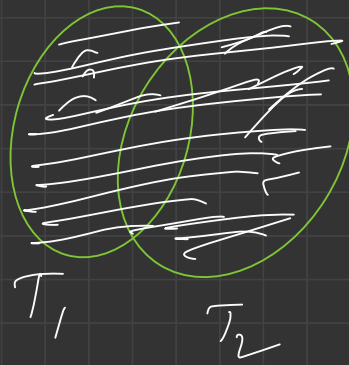
Syntax! →

Select * FROM Table 1

UNION

Select * FROM Table 2;

Set operation
→ Combine
two or more
select statements



JOIN vs UNION

Full Join :-

Col 1	Col 2	Col 1	Col 2
A	1	A	1
B	1	B	2
C	2	Null	Null
Null	Null	D	3

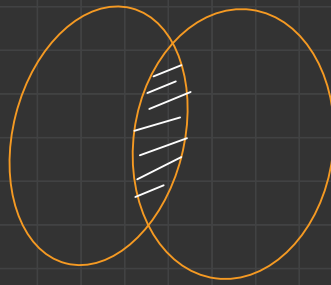
② INTERSECT

select * from T₁

INTERSECT

X

select * from T₂;



→ Emulate

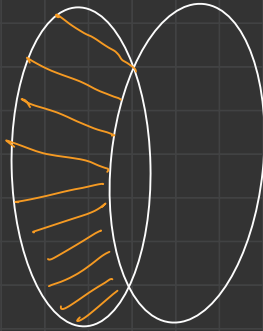
select DISTINCT id from T₁

INNER JOIN T₂

using (id);

③ MINUS

$T_1 - T_2$



Syntax:-> `SELECT id FROM T1
LEFT JOIN T2 USING(id)
WHERE T2.id IS NULL;`

Example

ABC

Dept 1

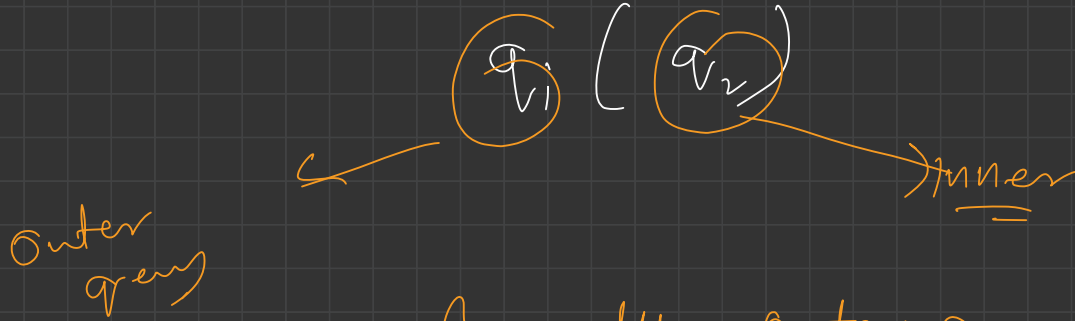
	empid	name	role
▶	1	A	engineer
	2	B	salesman
	3	C	manager
	4	D	salesman
	5	E	engineer
	NULL	NULL	NULL

Dept 2.

	empid	name	role
▶	3	C	manager
	6	F	marketing
	7	G	salesman
	NULL	NULL	NULL

Sub Queries

— alternative to joins



Generally, outer query depends on inner query

eg. $\text{select } * \text{ from table where id IN (select id from table where name = 'lat')};$

→ Work bench senior
* Project

* Project

	id	empID	name	startdate	clientID
▶	1	1	A	2021-04-21	3
	2	2	B	2021-03-12	1
	3	3	C	2021-01-16	5
	4	3	D	2021-04-27	2
	5	5	E	2021-05-01	4
	NULL	NULL	NULL	NULL	NULL

* EMPLOYEES

[illegible]

~~A~~ CLIENT

[illegible]

* Correlated Subquery

Outer (inner),

— inner query that refers the outer query =

```

-- Correlated subquery
-- find 3rd oldest employee
SELECT *
FROM Employee e1
WHERE 3 = (
    SELECT COUNT(e2.age)
    FROM Employee e2
    WHERE e2.age >= e1.age
);

```

for each, e1.age inner $\rightarrow$ completely one time

① e1.age = 32
 $\downarrow$ inner
 $\Rightarrow$ ②

② e1.age = 44
 $\Rightarrow \emptyset$

③ 22
 $\Rightarrow$ 4

32
44
22
31
21

④ e1.age = 31
 $\Rightarrow$ ③

SQL Views

⇒ MySQL → views

Customer ⇒ id | name | age | address |

view → name | age → alias — custom view

2) SQL

2) ~~AF~~ 50 Question Interview specific
ON SQL.

Home work

1
2

Note making
Practice